PUBLIC BUILD ARTIFACT

Agentic Fleet Build Guide

A practical cloud-first guide to build one useful AI employee, make it reliable, and grow a governed fleet without exposing private operating details.

START WITH ONE USEFUL JOB. EARN THE FLEET.

Roman Bediner | romanbediner.com/resources/agentic-ai-employees

12-page implementation companion

START HERE

Build one useful agent before you build a fleet.

A fleet is not eight chatbots with job titles. It is a set of bounded systems that can produce useful work, leave evidence, ask for a decision when they need one, and get better without silently changing themselves.

1. JOB

Pick one repeated outcome

Example: turn a defined cloud folder into one morning briefing. Avoid a broad role such as 'run marketing.'

2. HUMAN MOMENT

Choose the review surface

Use a Slack DM or email with a clear approve, reject, or revise action. The first version should make a human decision easy.

3. BOUNDARY

Name what cannot happen

State prohibited actions, spending limits, data limits, and when the run must stop and escalate.

4. PROOF

Define success before code

Write one measurable result, a visible completion record, and what a failed run should tell the operator.

Your first-agent canvas

INPUT

What cloud source arrives?

OUTPUT

What useful artifact is produced?

REVIEW

Who approves or corrects it?

EVIDENCE

Where is completion or failure recorded?

CLOUD-FIRST ARCHITECTURE

Use four layers. Keep the responsibility of each layer clear.

Choose managed cloud services your team can observe and maintain. The specific vendor can change. The contract between layers should not.

1

Behavior

PRD, job instructions, policies, and evaluation examples.

2

Runtime

Scheduled or event-driven code that reads inputs, runs tools, and records outcomes.

3

State

Shared memory, queue state, approvals, run ledger, and durable source records.

4

Surfaces

Slack, email, dashboards, APIs, and human review points where work is consumed.

Cloud-only baseline

Put source files, runtime, records, and integrations in managed cloud systems. A personal laptop can be a development tool, never the only place the agent can run or remember.

SHARED MEMORY

Hivemind turns shared context into a working loop.

Hivemind is the public name for the shared-memory pattern informing this fleet: retrieve the right context before work, capture the verified lesson after work, and let stale memory lose authority. It is credited here without linking out to a third party.

RECALL

Recall

Retrieve only the scoped, relevant instructions and prior evidence.

ACT

Act

Run the bounded job and cite the source material used.

CAPTURE

Capture

Write a compact lesson only after review or measurable confirmation.

Memory rules worth implementing

1. **Scope it**
   Separate company-wide context from a role, project, or customer context. Never retrieve every memory for every job.
2. **Ground it**
   Store source links, decision dates, and confidence. A remembered assertion is not the same thing as verified evidence.
3. **Correct it**
   Human rejection, a failed evaluation, or a new source should amend or supersede the old lesson.
4. **Let it fade**
   Reduce the influence of stale, unconfirmed context instead of letting old instructions silently govern new work.

PERFORMANCE AND COST

Treat model routing and caching as explicit product policy.

Do not let every task default to the most expensive model or recompute stable work. Put the decision in a small, reviewable policy layer and measure it by job.

ROUTING

Send the job to the right capability

Start deterministic when rules can solve it. Use a lighter model for extraction and classification. Escalate to a stronger model only for ambiguity, synthesis, or high-stakes review.

CACHING

Keep stable instructions stable

Separate a byte-stable system prefix from dynamic run data. Cache reusable context where your provider supports it. Match cache lifetime to the job cadence and invalidate when policy changes.

A simple routing decision

1. **Can a deterministic rule answer it?**
2. **If not, what is the cheapest model that meets the quality bar?**
3. **Is the output high-impact enough to require a stronger model or human review?**

Measure each job

Record quality signals, latency, token or request cost, cache hit rate, retries, and human correction rate. A model policy improves when it is observable, not when it is assumed.

INPUTS, TOOLS, AND REVIEW

Integrations should be narrow, observable, and easy to reverse.

The agent only needs the inputs and permissions required for its declared job. Add integrations after the first useful result, not before.

INPUT

Cloud sources

Use a cloud folder, structured table, inbox, or API. Record exactly which source records informed the output.

ACTION

A small tool set

Wrap external calls in clients with timeouts, permissions, idempotency, and human-readable errors.

REVIEW

Slack or email

Send the outcome to a human surface with clear approve, reject, and revise actions.

Integration contract

- Authenticate with service identities or scoped tokens, never a human fallback identity.
- Use idempotency keys or a durable state record before a write so retries cannot duplicate the action.
- Show what happened: source, timestamp, result, error category, and next owner.
- Make a failed integration legible: source returned nothing, permission issue, timeout, or provider outage.

Slack is a surface, not the system of record. Keep durable workflow state and completion evidence in a reliable cloud record even when Slack is where the human sees the work.

RELIABILITY

Nothing should fail silently. Design the recovery path before launch.

A run is not complete because a model returned text. It is complete when its output, side effect, and evidence record agree or a visible escalation owns the mismatch.

1

START

Create a run ID and record expected input and deadline.

2

DO

Execute with timeouts, bounded retries, and idempotent writes.

3

VERIFY

Read back the destination or check an independent success signal.

4

ESCALATE

Classify the failure, preserve evidence, and name the human or repair workflow.

Minimum run record

HEARTBEAT	last started, last completed, next expected run
RESULT	input references, output location, approval state
FAILURE	category, retry count, source response, owner
RECOVERY	retry safely, repair with review, or roll back

Use alerts for conditions that need attention, not every event. A useful alert says what failed, what evidence exists, what was attempted, and what should happen next.

SAFE IMPROVEMENT

A self-improving fleet still needs independent checks.

Learning becomes an engineering change only after a signal is captured, a change is reviewed, and production behavior is verified. No single agent should write, approve, and ship its own change.

SIGNAL

Signal

A human correction, failed run, or recurring friction becomes a clearly framed improvement ticket.

CHANGE

Change

A builder agent or engineer works in a branch with tests, updated documentation, and a rollback plan.

REVIEW

Review

An independent reviewer checks the proposed change against the PRD, tests, security, and user intent.

VERIFY

Verify

After release, observe the real behavior, confirm the intended evidence, then capture a reviewed memory.

The independence rule

Separate the authority to propose a change from the authority to approve and deploy it. Protect secrets, permissions, production deployment, and high-impact actions with review gates. This is how speed remains accountable.

QUALITY AND ECONOMICS

Make quality, cost, and trust visible in one operating view.

The objective is not maximum autonomy. It is reliable useful work at the appropriate cost, with a human able to see and correct the system.

QUALITY	acceptance rate, correction rate, evaluation pass rate
SPEED	time to useful output, queue age, missed schedule
COST	cost per completed job, model tier mix, cache hit rate
TRUST	approved changes, escalation clarity, recoverable failures

A better optimization question

Instead of asking 'How do we make the agent cheaper?', ask: 'Which part of this job creates value, which part needs judgment, and which part can be cached, simplified, or handled deterministically?'

BUILD CONTRACT

Write the PRD before you build. Keep it alive after you launch.

A lightweight PRD is the agent's operating contract. It prevents a vague ambition from becoming an untestable system and gives reviewers something concrete to protect.

Outcome and user

Who receives what useful result, when, and in what review surface?

Inputs and boundaries

What sources are allowed? What actions are prohibited or require approval?

Workflow and state

What happens in order, what is stored, and what makes a retry safe?

Success and failure

How is quality evaluated, what proves completion, and who owns recovery?

The documentation gate

When behavior, infrastructure, or a user-facing promise changes, update the PRD, operating diagram, and tests in the same change. A system without current documentation creates hidden risk for its next human or agent operator.

FIRST IMPLEMENTATION

The first ten moves from idea to a working AI employee.

1. **Choose the job**
   Pick one narrow, repeated job with a clear receiver.
2. **Write the contract**
   Create the one-page PRD and name the non-negotiable boundaries.
3. **Create the foundation**
   Set up a cloud repository, managed runtime, and durable source-of-truth record.
4. **Connect one input**
   Use one cloud input and store a representative evaluation set.
5. **Prove the flow**
   Build a deterministic version before adding a model where possible.
6. **Add model policy**
   Add routing policy, stable prompt structure, and cacheable context.
7. **Create the review**
   Send one draft to Slack or email with approval and correction controls.
8. **Make failures visible**
   Create a run ledger, heartbeats, readable errors, and safe retry behavior.
9. **Test the edge cases**
   Test the happy path, empty input, bad input, provider failure, and duplicate retry.
10. **Pilot and learn**
    Launch a limited pilot, review corrections, and turn verified lessons into backlog items.

RELEASE CHECKLIST

Do not call it autonomous until you can answer these questions.

Use this as a release conversation with the operator, engineer, and reviewer. If an answer is unknown, it is a build task, not an acceptable assumption.

USEFUL

- ☐ Can a real user name the job and recognize a good result?
- ☐ Is there a smaller first version that delivers value sooner?

BOUNDED

- ☐ Are data access, actions, approvals, and escalation limits explicit?
- ☐ Can the system fail closed instead of impersonating a human or guessing?

OBSERVABLE

- ☐ Can an operator see the last run, current state, and evidence of the result?
- ☐ Does every failure name an owner and next action?

MAINTAINABLE

- ☐ Are PRD, diagrams, tests, deployment, and rollback instructions current?
- ☐ Does an independent reviewer gate meaningful changes?

Build the next role only when the first role creates a real, visible need for it.

That is how a project manager creates the case for an improvement engineer, a reviewer, or an orchestrator: through evidence, not org-chart theater.